ML25/26-11_ImplementReminderAgent

TechTrio · Sprint 1: 1–14 June 2026

We are submitting this document to confirm that our understanding and approach for the cloud migration are correct before we proceed with implementation. We have four specific questions at the end where we need guidance.

1. How We Map MyCloudProjectSample to Our Project

MyCloudProjectSample	ReminderAgent.Cloud equivalent
MyCloudProject/Program.cs	ReminderAgent.Cloud/Program.cs — single entry point, DI + queue loop
MyCloudProject.Common/ (IExperiment, IExperimentResult, IExerimentRequest, InitHelpers)	ReminderAgent.Common/ (IReminderAgent, IReminderResult, ReminderRequest, InitHelpers) + existing interfaces copied verbatim
MyExperiment/Experiment.cs	ReminderTools.cs — copied verbatim, no changes to business logic
MyExperiment/ExperimentResult.cs	InteractionResult.cs — ITableEntity row written to Interactions table
MyExperiment/ExerimentRequestMessage.cs	ReminderRequest.cs — deserialized queue message
MyExperiment/AzureStorageProvider.cs	Split into 3 files — see section 2
MyExperiment/MyConfig.cs	MyConfig.cs — same structure, extended with our container/table names

2. Why AzureStorageProvider Is Split Into Three Files

The original ReminderAgent already had two separate interfaces — IStorageProvider (asset CRUD) and IEmbeddingStore (vector read/write) — because the two stores have different schemas and a write-ordering invariant between them. Merging them back into one class breaks that boundary. The three files map directly to AzureStorageProvider:

File	Replaces	Interface
AzureBlobStorageProvider.cs	CsvStorageProvider + reminders.csv, JsonEmbeddingStore + embeddings.json	IStorageProvider, IEmbeddingStore
AzureQueueHandler.cs	REPL loop in original Program.cs	Called from Program.cs directly

AzureTableStorage.cs → Mirrors ExperimentResult

3. Azure Storage Design

- Blob Storage - replaces reminders.csv and embeddings— partition key: "embeddings", row key: asset GUID, property: Vector (Base64 float[1536]). Replaces embeddings.json.
- Table Storage holds - Interactions(Result) — partition key: date, row key: request GUID. Stores InputMessage, ToolCalled, ResponseText, DurationMs, Timestamp. Mirrors ExperimentResult — satisfies the course requirement of writing results to Table Storage.

4. What Stays the Same vs What Changes

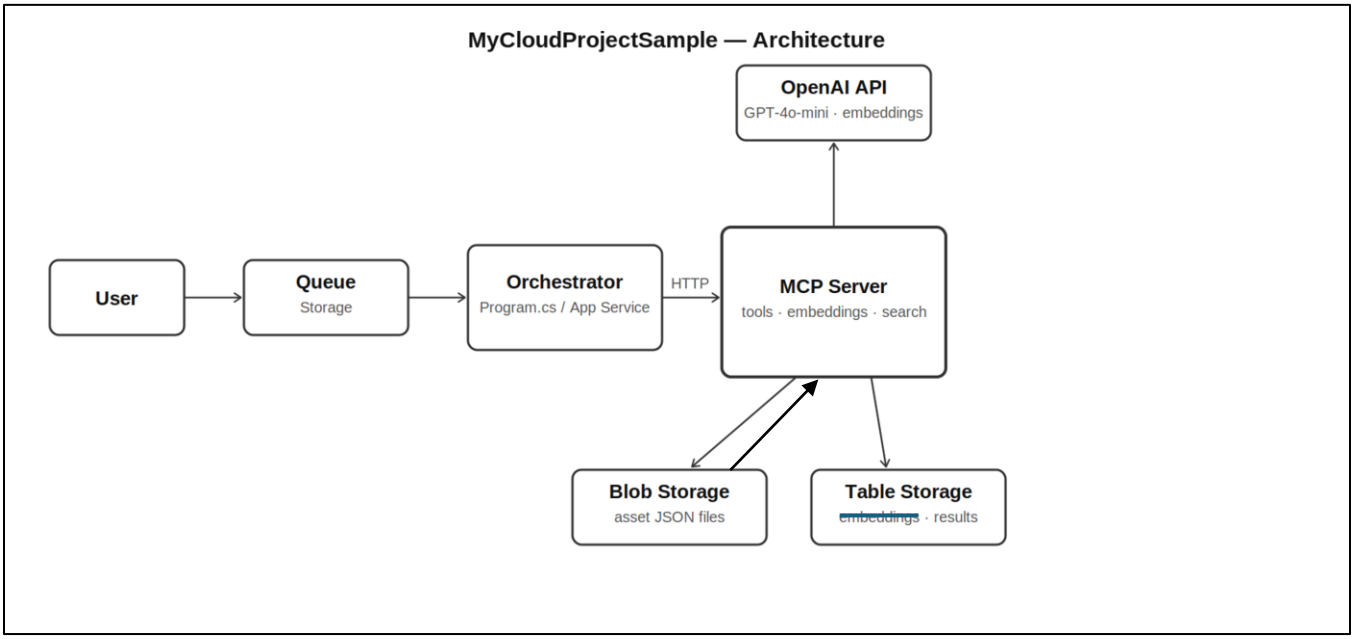
Copied verbatim — no changes:

- All 7 ReminderTools methods, EmbeddingService, SimilarityService, all 4 interfaces, Asset.cs

Changed at infrastructure layer only:

- CsvStorageProvider, JsonEmbeddingStore → AzureBlobStorageProvider (one blob per asset, ETag concurrency replaces SemaphoreSlim)
- Adding Interactions (ExperimentResult)→ AzureTableEmbeddingStore
- File logging → stdout
- Synchronous REPL → Azure Queue poll loop (500ms interval, mirrors course template)

5. Architecture Summary



The system has two deployed containers on Azure App Service, one external API, and three Azure storage services.

Orchestrator container is the entry point. It runs Program.cs which polls Queue Storage every 500ms. Its only job is to receive a user message, forward it to the MCP Server over HTTP, write the interaction result to Table Storage, and delete the message from the queue. It contains no business logic.

MCP Server container is where all intelligence lives. It holds ReminderTools (7 tool methods), EmbeddingService (vector generation), and SimilarityService (cosine scoring). It is the only container that talks to Blob Storage, Table Storage, and OpenAI. It receives requests from the Orchestrator over HTTP following the MCP protocol and returns tool results.

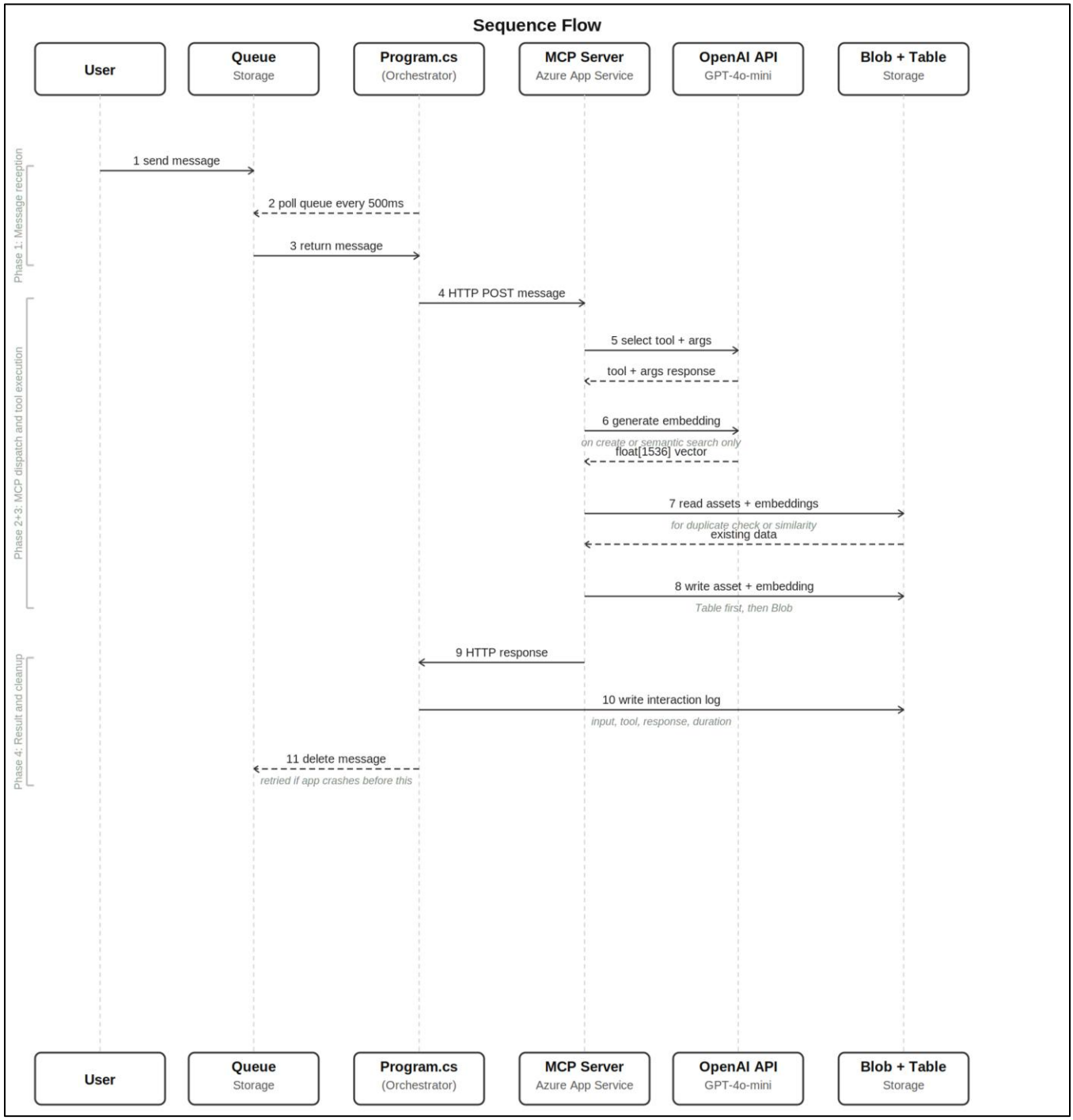
Queue Storage is the entry point for user messages. It decouples the user from the processing — if the app is busy or restarts, the message waits safely in the queue and is never lost.

Blob Storage holds all assets as individual JSON blobs, one per asset, keyed by GUID. This replaces the original reminders.csv. Each asset is independently addressable and there are no read-modify-write conflicts. The Embeddings table stores float [1536] vectors keyed by asset GUID, replacing the original embeddings.json

Table Storage holds - The Interactions (Result) table stores one row per user request — input, tool called, response, timestamp, duration — this is the result log required by the course template.

OpenAI API is called by the MCP Server only, never by the Orchestrator. It serves two purposes: GPT-4o-mini selects which tool to invoke and extracts arguments from the user's natural language, and text-embedding-3-small generates the 1,536-dimensional vectors used for semantic search.

6. Sequency Flow Summary



Step	Who does it	What happens
1	User → Queue Storage	User types a message. It is sent into Azure Queue Storage.
2	Program.cs → Queue Storage	Program.cs polls the queue every 500ms asking "is there a message?"
3	Queue Storage → Program.cs	Queue returns the user's message to Program.cs
4	Program.cs → MCP Server	Program.cs forwards the message to the MCP Server via HTTP POST
5	MCP Server → OpenAI → MCP Server	GPT-4o-mini reads the message, picks which tool to call and extracts the arguments
6	MCP Server → OpenAI → MCP Server	Embedding model generates float[1536] vector for the message (only on create or semantic search)
7	MCP Server → Blob	MCP Server reads existing assets and existing embeddings from Blob
8	MCP Server → Blob	MCP Server writes the new asset and embeddings to Blob Storage
9	MCP Server → Program.cs	MCP Server sends the HTTP response back to Program.cs with the result
10	Program.cs → Table Storage	Program.cs writes the interaction record — input, tool used, response, duration — to Table Storage
11	Program.cs → Queue Storage	Program.cs deletes the message from the queue confirming successful processing

7. File Structure

```
ReminderAgent.Cloud/                    ← solution root
│
├── ReminderAgent.Cloud/                ← console app (Docker target)
│   ├── Program.cs                      ← DI wiring + queue poll loop
│   ├── appsettings.json
│   ├── appsettings.dev.json
│   └── Dockerfile
│
├── ReminderAgent.Common/               ← mirrors MyCloudProject.Common
│   ├── Asset.cs                        ← copied verbatim from original
│   ├── IStorageProvider.cs             ← copied verbatim from original
│   ├── IEmbeddingStore.cs              ← copied verbatim from original
│   ├── IEmbeddingService.cs            ← copied verbatim from original
│   ├── ISimilarityService.cs           ← copied verbatim from original
│   ├── IReminderAgent.cs               ← mirrors IExperiment.cs
│   ├── IReminderResult.cs              ← mirrors IExperimentResult.cs
│   ├── ReminderRequest.cs              ← mirrors IExerimentRequest.cs
│   └── InitHelpers.cs                  ← mirrors InitHelpers.cs
│
└── ReminderAgent.Infrastructure/       ← mirrors MyExperiment
    ├── AzureBlobStorageProvider.cs     ← implements IStorageProvider
    ├── AzureTableEmbeddingStore.cs     ← implements IEmbeddingStore
    ├── AzureQueueHandler.cs            ← queue receive + commit
    ├── ReminderTools.cs                ← mirrors Experiment.cs
    ├── InteractionResult.cs            ← mirrors ExperimentResult.cs
    ├── EmbeddingService.cs             ← copied verbatim from original
    ├── SimilarityService.cs            ← copied verbatim from original
    └── MyConfig.cs                     ← mirrors MyConfig.cs
```

7. Questions for Prof. Dobric

> *Q1 — Template mapping: We mapped IExperiment → IReminderAgent implemented by ReminderTools. Is this correct, or should ReminderTools remain without a new wrapper interface?*
>
> *Q2 — AzureStorageProvider split: We split it into three files justified by existing interface boundaries. Is this acceptable, or must we keep a single AzureStorageProvider class to match the template?*
>
> *Q3 — Table Storage: results (mirrors ExperimentResult). Is this the correct interpretation?*
>
> *Q4 — Is the File Structure correct – do we need to rename any file and folder those are in?*

TechTrio — Aishwarya Rajendra Sawant · KavyaSree Thabjul · Bavana Jadala · June 2026